Before You Arrive

The mathematics and programming you should revise before Week 1 of the workshop.

This is not new material. It is a checklist of things the course *assumes* you already know, so we never spend lecture time re-teaching basics. Everything here maps to something concrete you will use — most of it in the very first week.

"If you remember one thing from this whole course, remember the shape of this:"

$$V(s) = r + \gamma V(s')$$

How to use this document

Each topic follows the same rhythm: a plain-English **intuition**, the **equation** (annotated, with colours that we reuse in every lecture), a **worked example** tied to reinforcement learning, and a short **practice** problem with a full solution. Read actively — do the practice problems with a pen.

We colour the recurring quantities the same way throughout the course, so the notation becomes muscle memory:

value (what we want to learn)
reward (feedback signal)
discount γ (how much the future counts)
state (where the agent is)
action (what the agent does)

What to revise, and when you'll need it

You do *not* need all of this for day one. Prioritise by the tags:

Part	Needed by	Why
1 · Notation fluency	**Week 1**	You must read $V(s)$, $\max_a$, $\sum$, $\mathbb{E}[\cdot]$ on sight.
2 · Probability & statistics	**Week 1**	The core. Value is an *expectation*; GRPO uses *z-scores*.
3 · A little calculus	Week 3	Gradients arrive with policy gradients (REINFORCE/PPO).
4 · A little linear algebra	Week 2	To read a DQN / PPO network.
5 · A little information theory	Week 3	KL divergence is the "leash" in PPO / RLHF / GRPO.
6 · Programming readiness	**Week 1** (Python), Week 2 (PyTorch)	You code from day one.
7 · Systems literacy	Week 4	GPUs, tokens, memory — for the inference half.
8 · Warm-up: watch/read/play	Anytime	Build intuition before the math.
9 · Self-diagnostic	Before Week 1	15 questions. Find your gaps.

Contents

1 Notation fluency

Needed: Week 1

Most people who "find the math hard" actually just find the *notation* unfamiliar. The ideas are simple; the symbols are dense. This part is a decoder ring. If you can read every line below out loud in plain English, you are ready.

1.1 Function notation: V(s) and Q(s,a)

> $V(s)$ is read "V of s" — a function that takes a state s and returns a number. Think of it as a lookup: give it where you are, it tells you how good that is. $Q(s,a)$ takes *two* inputs — a state and an action — and returns how good it is to take action a while in state s.

A superscript names *which policy* we are measuring: $V^{\pi}(s)$ is the value of state s *if you act according to policy* π. A star means *optimal*: $V^{*}(s)$ is the best achievable value. A prime means *next*: s' is "the state you land in after this one."

1.2 Summation (the sigma symbol)

> $\sum$ (capital sigma) just means "add these up." $\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$. The thing under the $\sum$ says where to start and what the index is; the thing on top says where to stop.

You will often see sums over states, $\sum_{s'} \ldots$, which means "add this up over every possible next state."

1.3 Maximum and argmax

> $\max_a f(a)$ is the *largest value* f takes as you try every action a. $\arg\max_a f(a)$ is the *action that achieves* that largest value. One gives you the height of the peak; the other gives you where the peak is. In RL, max tells you how good the best move is; arg max tells you which move to make.

1.4 The expectation operator

This is the single most important symbol in the course; Part 2 teaches it properly. For now: $\mathbb{E}[X]$ means "the average value of X over all the randomness," and $\mathbb{E}[X \mid Y]$ means "the average of X, given that we know Y."

1.5 Reading a recursive equation

Putting it together — here is the equation from the title page, read symbol by symbol:

$$V(s) \;=\; r \;+\; \gamma\, V(s')$$

"The value of where I am equals the reward I just received plus the discount times the value of where I land next." It is *recursive*: V appears on both sides. That is not circular — it is the definition that the whole course learns how to solve.

This single line, rearranged, *is* every algorithm in the course. DQN, PPO, RLHF, and GRPO are all different ways of computing the right-hand side when the state space is too big, the model is unknown, or the reward arrives late.

Read these aloud in plain English: (a) $\max_a Q^*(s,a)$ (b) $\sum_{s'} P(s' \mid s,a)\, V(s')$ (c) $\arg\max_a Q(s,a)$.

Solution. (a) "the value of the best action available in state s, under the optimal policy." (b) "the average next-state value, weighting each possible next state s' by how likely action a is to take you there." (c) "the action that is best in state s" — i.e. what to actually do.

2 Probability & statistics

Needed: Week 1 – the core

If you revise only one part, revise this one. Reinforcement learning is built on the idea that the world is uncertain and that "how good" a situation is means "how good *on average*."

2.1 Random variables and distributions

A *random variable* is a quantity whose value depends on chance — the roll of a die, tomorrow's reward, the next state the environment puts you in. A *probability distribution* lists each possible value and how likely it is. For a fair die: each of $\{1,2,3,4,5,6\}$ has probability $\frac{1}{6}$.

Probabilities are never negative and always sum to 1: $\sum_x P(x) = 1$.

2.2 Expectation: the average outcome

The *expectation* $\mathbb{E}[X]$ is the long-run average of a random variable, weighting each outcome by its probability. It is the centre of gravity of the distribution.

$$\mathbb{E}[X] \;=\; \sum_x x \cdot P(x).$$

A fair die: $\mathbb{E}[X] = 1\cdot\frac{1}{6} + 2\cdot\frac{1}{6} + \cdots + 6\cdot\frac{1}{6} = \frac{21}{6} = 3.5$. You never roll a 3.5 — but it is the average over many rolls.

Now the RL version. The *return* G_t is the total (discounted) reward from time t onward — a random number, because the environment and the policy are stochastic. The value of a state is the *expected* return:

$$V^\pi(s) \;=\; \mathbb{E}_\pi[\, G_t \mid s_t = s \,].$$

"Starting from state s and following policy π, what total reward should I expect on average?" The subscript π on $\mathbb{E}$ says the randomness includes the policy's own choices.

2.3 Conditional probability and the Markov property

$P(A \mid B)$ is "the probability of A *given that* B happened." Knowing B can change the odds of A.

A process has the *Markov property* when the next state depends only on the current state and action — not on the entire history:

$$P(s_{t+1} \mid s_t, a_t) \;=\; P(s_{t+1} \mid s_t, a_t, s_{t-1}, \ldots, s_0).$$

The Markov property is the assumption that lets us write $V(s)$ as a function of s *alone*. If the future depended on the whole past, value could not be a simple lookup on the current state, and none of the recursion would work.

2.4 Variance, standard deviation, and the z-score

The *variance* measures how spread out a random variable is around its mean; the *standard deviation* σ is its square root (same units as the data). A *z-score* re-expresses a value as "how many standard deviations above or below the mean" it is.

$$\mathrm{Var}[X] = \mathbb{E}\big[(X-\mu)^2\big], \qquad \sigma = \sqrt{\mathrm{Var}[X]}, \qquad z = \frac{x-\mu}{\sigma}.$$

GRPO — the algorithm behind modern reasoning models, and a centrepiece of Week 3 — turns each sampled return into an *advantage* using exactly the z-score:

$$A_i \;=\; \frac{R_i - \mathrm{mean}(R)}{\mathrm{std}(R)}.$$

A rollout that scored well *relative to its group* gets a positive advantage and is reinforced. No separate value network needed — the group's own mean and standard deviation provide the baseline.

Four rollouts score $R = \{2, 8, 6, 4\}$. Compute the mean, the (population) standard deviation, and the z-score (advantage) of the rollout that scored 8.

Solution. Mean $= (2+8+6+4)/4 = 5$. Squared deviations: $9, 9, 1, 1$; variance $= 20/4 = 5$; $\sigma = \sqrt{5} \approx 2.24$. Advantage of the 8: $z = (8-5)/2.24 \approx \mathbf{+1.34}$. It scored well above its group, so it is reinforced.

2.5 Sampling and Monte-Carlo estimation

When you cannot compute an expectation exactly (the formula is intractable, or you do not even know the distribution), you *estimate* it: draw many samples and average them. By the law of large numbers, the sample average converges to the true expectation as you collect more samples.

This is the entire idea behind Monte-Carlo RL: the agent cannot compute $\mathbb{E}[G_t]$ directly, so it plays many episodes and averages the returns it actually observes. "Monte Carlo" just means "estimate an expectation by sampling."

2.6 The running average – the shape of every RL update

You can update an average *incrementally*, without storing all the data, by nudging the old estimate a small step toward each new observation:

$$\mu_{\text{new}} = \mu_{\text{old}} + \alpha\,(x - \mu_{\text{old}}).$$

Here α is a small step size and $(x - \mu_{\text{old}})$ is the "error" — how far the new data point is from the current estimate.

Look at the temporal-difference update that powers Q-learning, DQN, and the critic in actor-critic:

$$V(s) \leftarrow V(s) + \alpha[\, r + \gamma V(s') - V(s)\,].$$

It is *exactly* a running average. The bracket is the error (the "TD error"); the agent nudges its estimate a step α toward $r + \gamma V(s')$. Recognising this shape means you already understand the core of modern RL.

2.7 Geometric series – why discounting works

A geometric series adds up powers of a ratio: $1 + r + r^2 + r^3 + \ldots$. When $|r| < 1$ the terms shrink fast enough that the infinite sum is *finite*.

$$\sum_{k=0}^{\infty} r^k = \frac{1}{1-r}, \qquad \text{for } |r| < 1.$$

The return discounts future reward by γ per step: $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots$. If every reward were 1, the total would be $\sum_k \gamma^k = \frac{1}{1-\gamma}$ — a finite number because $\gamma < 1$. That finiteness is exactly what makes value well-defined over an infinite horizon. With $\gamma = 0.99$,

the effective horizon is about $\frac{1}{1-0.99} = 100$ steps.

With $\gamma = 0.9$ and a reward of $+1$ at every step forever, what is the total discounted return? What if $\gamma = 0.5$?

Solution. $\sum_k 0.9^k = \frac{1}{1-0.9} = \mathbf{10}$. $\sum_k 0.5^k = \frac{1}{1-0.5} = \mathbf{2}$. A more patient agent (higher γ) values the same reward stream much more.

3 A little calculus

Needed: Week 3

You need surprisingly little calculus, and none of it for Week 1. The goal is to understand *gradients* — the direction to nudge parameters to improve a model.

3.1 Derivative and gradient

A *derivative* $\frac{df}{dx}$ is the slope of f at a point: how fast f changes as you nudge x. A *gradient* $\nabla_\theta f$ is just the collection of partial derivatives when f depends on many parameters θ — it points in the direction of steepest increase.

3.2 Gradient ascent / descent

To *maximise* a function, repeatedly step in the direction of its gradient (uphill): $\theta \leftarrow \theta + \alpha \nabla_\theta f$. To *minimise* (e.g. a loss), step against it: $\theta \leftarrow \theta - \alpha \nabla_\theta L$. This hill-climbing is how every neural network in the course is trained.

3.3 The chain rule (conceptually)

The chain rule says the derivative of nested functions multiplies: if $y = f(g(x))$, then $\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$. You will never compute this by hand — *backpropagation* is the chain rule applied automatically through a network — but you should know that is what PyTorch's `.backward()` is doing.

3.4 The logarithm and its derivative

$\log(x)$ is the inverse of exponentiation, and it turns products into sums: $\log(ab) = \log a + \log b$. Its derivative is clean: $\frac{d}{dx} \log x = \frac{1}{x}$.

Policy-gradient methods (REINFORCE, PPO, RLHF) optimise the *log-probability* of good actions, $\log \pi(a \mid s)$, because logs turn the product of probabilities along a trajectory into a sum — far easier to differentiate. When you see $\nabla_\theta \log \pi$ in Week 3, this is why.

4 A little linear algebra

Needed: Week 2

4.1 Vectors and the dot product

A *vector* is an ordered list of numbers — e.g. a state encoded as $[0.2, -1.1, 3.0]$. The *dot product* multiplies two vectors element-wise and sums the result: $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$. It measures how aligned two vectors are, and it is the basic operation inside every neural-network layer.

4.2 Matrix–vector multiplication = one network layer

A matrix is a grid of numbers; multiplying a matrix W by a vector $\mathbf{x}$ produces a new vector whose entries are dot products of $\mathbf{x}$ with each row of W. That is *exactly* what one layer of a neural network computes: $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, followed by a nonlinearity.

When DQN replaces the value *table* with a value *network* (Week 2), $V(s)$ becomes a stack of these matrix multiplies. You do not need eigenvalues or matrix inverses for this course — just the picture of "a layer is a matrix multiply plus a bias."

5 A little information theory

Needed: Week 3

5.1 Log-probability and entropy

A rare event carries more "information" than a common one — $-\log P(x)$ is large when $P(x)$ is small. *Entropy* is the average information of a distribution, $H = -\sum_x P(x) \log P(x)$; it is highest when outcomes are most uncertain (uniform) and zero when one outcome is certain. In RL, an *entropy bonus* is sometimes added to keep a policy exploring rather than collapsing too early.

5.2 KL divergence – the "leash"

The *Kullback–Leibler divergence* $D_{\mathrm{KL}}(\pi_1 \| \pi_2)$ measures how different two probability distributions are. It is 0 when they are identical and grows as they diverge (it is not symmetric — order matters).

$$D_{\mathrm{KL}}(\pi_1 \,\|\, \pi_2) \;=\; \mathbb{E}_{\pi_1}\big[\log \pi_1(a \mid s) - \log \pi_2(a \mid s)\big].$$

KL divergence is the safety leash in almost every modern policy method. TRPO constrains each update to a small KL ball; PPO approximates the same idea by clipping; RLHF and GRPO add a KL penalty against the original model ($\beta\, D_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})$) so the fine-tuned policy does not drift into gibberish while chasing reward. Whenever you hear "don't let the policy move too far," the mechanism is KL.

6 Programming readiness

Needed: Python: Week 1 · PyTorch: Week 2

You will write code from the first session. None of it is exotic, but you should be fluent in the following without looking things up.

6.1 Python you must be fluent in

- **Dictionaries as sparse tables.** The value function starts life as a dict: `V.get(s, 0.0)` returns the stored value or a default of 0 for an unseen state. This is the data structure behind the tic-tac-toe agents.
- **Tuples** as immutable keys (a board state is a 9-tuple), **list comprehensions** (`[a for a in legal_actions(s)]`), **type hints** (`Dict[State, float]`), and basic **classes**.
- **Control flow**: `while`/`for` loops, `if`/`break` for terminal conditions.
- **Seeded randomness**: `rng = random.Random(42)`, `rng.choice(options)` — for reproducible sampling.

6.2 NumPy essentials

- Creating arrays, element-wise arithmetic, and *vectorised* operations (no Python loop).
- `arr.mean()`, `arr.std()`, `np.argmax(arr)` — the z-score / advantage computation is three NumPy calls.

6.3 PyTorch essentials (for Week 2 onward)

You should recognise the four moving parts of a training step. In pseudocode:

```
pred = model(x)   # forward pass
loss = loss_fn(pred, target)
optimizer.zero_grad()  # clear old gradients
loss.backward()     # chain rule, automatically
optimizer.step()     # nudge parameters downhill
```

Know what a **tensor** is (an array that tracks gradients), what `nn.Module` is (a container for layers), and what `autograd` does (computes gradients via the chain rule so you never differentiate by hand).

6.4 Environment setup – do this before Week 1

- Create an isolated environment: `python -m venv .venv && source .venv/bin/activate` (or conda).
- Install `numpy` and `torch`.
- Confirm GPU access if you have one: `python -c "import torch; print(torch.cuda.is_available())"`. Don't worry if it prints `False` — the early weeks run fine on CPU.

7 Systems & infrastructure literacy

Needed: Week 4 – the production half

Conceptual only — no setup required now. The second half of the course is about *serving* RL systems, so a working mental model of the hardware helps.

- **GPU and VRAM.** A GPU does many multiply-adds in parallel; its memory (VRAM) holds the model weights and the intermediate activations. Model size in VRAM ≈ (number of parameters) × (bytes per parameter). A 4-billion-parameter model in 16-bit precision needs ≈ 8 GB just for weights.
- **FLOPs and bandwidth.** "How fast" splits into compute (floating-point ops per second) and memory bandwidth (how fast you can move data). We will see that generating tokens one at a time is limited by *bandwidth*, not compute — which shapes every optimisation.
- **Tokens and context windows.** Language models read and write *tokens* (sub-word chunks). The *context window* is how many tokens the model can attend to at once. Cost scales with tokens processed — a fact we will turn into a first-principles cost model.

8 Warm-up: watch, read, play

Needed: Anytime

Build intuition *before* the math hardens it. In rough order of effort:

- **Play (15 min, no math).** Open the tic-tac-toe visualiser shipped in this repo (`code/tic-tac-toe/visualizer/`). Watch Dynamic Programming, Monte Carlo, and Temporal Difference all learn the same game side by side, then play the trained agent. You will *see* the ideas of Part 2 before you read the equations.
- **Watch.** David Silver's *RL Course*, Lecture 1 (DeepMind, on YouTube) — the cleanest spoken introduction to MDPs and value. Karpathy's blog post *"Deep Reinforcement Learning: Pong from Pixels"* for the policy-gradient intuition.

- **Read.** Sutton & Barto, *Reinforcement Learning: An Introduction* (free PDF), Chapters 3 (MDPs), 4 (Dynamic Programming), 5 (Monte Carlo), 6 (Temporal-Difference). These four chapters *are* Lecture 1. The curated paper list lives in `resources/papers.md`.

9 "Are you ready?" – self-diagnostic

Needed: Before Week 1

Fifteen questions. Work them with a pen, then check the answer key. There is no grade — this is a map of your own gaps. Scoring guidance follows the key.

Questions

Q1. *(Notation)* In plain English, what does $\arg\max_a Q(s,a)$ give you, and how does it differ from $\max_a Q(s,a)$?

Q2. *(Notation)* Read aloud: $\sum_{s'} P(s' \mid s,a)\, V(s')$.

Q3. *(Notation)* What is the difference between $V^{\pi}(s)$ and $V^{*}(s)$?

Q4. *(Probability)* A fair four-sided die shows $\{1,2,3,4\}$. What is $\mathbb{E}[X]$?

Q5. *(Probability)* In one sentence, state the Markov property and why it matters for defining $V(s)$.

Q6. *(Probability)* Rewards are $\{10,0,5,5\}$. Give the mean, population standard deviation, and the z-score of the 10.

Q7. *(Probability)* With $\gamma = 0.95$ and reward $+1$ at every step forever, what is the total discounted return?

Q8. *(Probability)* Why does the temporal-difference update $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$ have the shape of a running average?

Q9. *(Calculus)* What does the gradient $\nabla_\theta L$ point toward, and which way do you step to *minimise* L?

Q10. *(Calculus)* Why do policy-gradient methods work with $\log \pi(a \mid s)$ rather than $\pi(a \mid s)$ directly?

Q11. *(Linear algebra)* Write the operation one neural-network layer performs on an input vector $\mathbf{x}$.

Q12. *(Information theory)* What does $D_{\mathrm{KL}}(\pi_1 \| \pi_2) = 0$ tell you about the two policies?

Q13. *(Information theory)* In one phrase, what job does the KL term do in PPO / RLHF / GRPO?

Q14. *(Programming)* What does `V.get(s, 0.0)` return, and why is the default useful in a value table?

Q15. *(Programming)* Name the four lines of a PyTorch training step, in order.

Answer key

A1. arg max returns the *action* that is best; max returns the best *value*. One is "which move," the other is "how good."

A2. "The average next-state value, weighting each possible next state s' by the probability that action a leads there."

A3. V^{π} is the value if you follow a *specific* policy π; V^{*} is the value under the *best possible* policy.

A4. $\mathbb{E}[X] = (1+2+3+4)/4 = 2.5$.

A5. The next state depends only on the current state and action, not the full history — so value can be a function of the current state alone.

A6. Mean $= 5$; deviations $\{5, -5, 0, 0\}$, variance $= 50/4 = 12.5$, $\sigma \approx 3.54$; z-score of the 10 is $(10-5)/3.54 \approx +1.41$.

A7. $\sum_k 0.95^k = 1/(1-0.95) = 20$.

A8. The bracket is an error term $(x - \text{old})$ with $x = r + \gamma V(s')$, and the update nudges the estimate a step α toward it — the incremental-average form $\mu \leftarrow \mu + \alpha(x - \mu)$.

A9. It points in the direction of steepest *increase* of L; to minimise, step in the *opposite* direction, $\theta \leftarrow \theta - \alpha \nabla_\theta L$.

A10. Logs turn the product of per-step probabilities along a trajectory into a sum, which is far easier to differentiate; and the gradient of $\log \pi$ has a clean, low-variance form.

A11. $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, followed by a nonlinearity — a matrix multiply plus a bias.

A12. The two policies are identical (KL is zero only when the distributions match exactly).

A13. It is the "leash" that keeps the updated policy from drifting too far from the reference/previous policy.

A14. It returns the stored value for state s, or 0.0 if s has never been seen — so you can treat unvisited states as value-0 without pre-filling the table.

A15. `pred = model(x)`; `loss = loss_fn(pred, target)`; `optimizer.zero_grad()`; `loss.backward()`; `optimizer.step()`. (Four-to-five lines; the key ones are forward, zero_grad, backward, step.)

How to read your score

- **Missed several in Q4–Q8 (probability)?** This is the one that matters most for Week 1. Reread Part 2 carefully and redo the practice problems — value functions, GRPO advantages, and discounting all live here.
- **Missed Q1–Q3 (notation)?** Spend an hour with Part 1 until the symbols read as plainly as English. Everything downstream depends on it.
- **Missed Q9–Q13 (calculus / linear algebra / information theory)?** No emergency — these arrive in Weeks 2–3. Skim now, revisit before those weeks.
- **Missed Q14–Q15 (programming)?** Work through the tic-tac-toe code in `code/tic-tac-toe/` and a short PyTorch "train a tiny net" tutorial before the first session.

See you in Week 1. Questions before then: hello@vizuara.com